Interior Design Client Management Portal

Project Specification Document

---

1. Overview

An agentic dashboard/portal for an interior design company that automates the client journey — from initial conversation to final quotation — replacing the current manual, paper-based process of noting requirements, tracking changes, and negotiating quotations.

The system combines:

- An **n8n-based agentic pipeline** that ingests client conversations, summarizes them, tracks changes over time via checkpoints, and eventually produces a quotation.
- A **web portal** (Next.js + FastAPI) where admins and designers manage clients, review checkpoint history, track project phases, and oversee quotations.

The pipeline's scope ends once a quotation is approved. Implementation (physical work) is tracked in the portal but not automated.

---

2. Problem Statement

Currently, interior designers:

1. Discuss requirements with clients manually (in person/calls), noting details on paper.
2. Requirements change iteratively (e.g., color, material) with no structured record of what changed and when.
3. Eventually reach agreement, select a design, and manually calculate a budget quotation.
4. Once agreed, implementation begins — with no centralized tracking of project stage or history.

This is slow, error-prone, hard to audit, and has no single source of truth for what was agreed at any point in time.

---

3. Solution Summary

- Conversations are fed into an **n8n pipeline** which summarizes them and detects what changed since the last summary.
- Each round of changes creates a **checkpoint** — a snapshot with a summary, a structured diff of what changed, and a status.
- Designers/admins can review the full checkpoint history for a client at any time (not just the latest state).
- Once client and designer agree, a **final summary** is drafted and sent to the client for confirmation.
- Once confirmed, **3-4 designs** are generated (AI-generated and/or designer-uploaded), each with structured material/dimension specs.
- Once a design is selected, the system **calculates a quotation** using designer-supplied reference data (material costs, labor rates, margins) applied against the selected design's specs.
- Once the quotation is approved, the project moves to **Implementation**, tracked manually in the portal, and finally to **Completed/Payment**.

---

4. User Roles

Admin

- Sees **all clients/projects** across the company.
- Can view any client's checkpoint history, selected design, quotation, and assigned designer.
- Can see which phase every project is in.
- Can override phase transitions and confirmations.
- Manages the materials/rates table and margin configuration.

Designer (Individual)

- Sees only clients **assigned to them**.
- Same detail view as admin, but scoped to their own clients.
- Can manually mark a client's final summary as confirmed (in case the client agreed verbally/offline instead of replying digitally).
- Uploads designs with manually entered specs, or reviews AI-generated design options.

There is **no separate client login**. Clients interact only via conversation (chat/call, parsed by n8n) and by replying to the final summary sent to them. Confirmation can happen two ways:

- **Client reply** — parsed by n8n for confirmation intent.
- **Designer override** — designer manually marks it confirmed if agreement happened verbally/offline.

Both paths are logged with who/what confirmed it, for accountability.

---

5. Project Phases

The project/client record moves through **5 phases**:

1. **Discussion** — Conversations happen, checkpoints are created iteratively as requirements evolve.
2. **Design** — 3-4 designs (AI-generated and/or designer-uploaded) are presented; one is selected.
3. **Quotation/Approval** — System calculates a cost breakdown for the selected design; client approves.
4. **Implementation** — Physical work begins; tracked manually in the portal (not automated).
5. **Completed/Payment** — Project and payment finalized.

Phase transitions are automatic by default (e.g., quotation approved → auto-move to Implementation), but **admin or assigned designer can manually override** the phase at any point.

---

6. Checkpoint System

Checkpoints are the core innovation of this system — instead of overwriting requirements every time something changes, each round of discussion produces a new checkpoint that preserves history.

Each checkpoint contains:

- **Summary** — a generated natural-language summary of the discussion at this point.
- **Structured diff** — what changed since the last checkpoint (e.g., `color: cream → pink`).
- **Status** — `draft`, `under_review`, or `agreed`.
- **Source** — whether it came from a parsed conversation or a manual entry.
- **Timestamp**.

Users can browse **any past checkpoint**, not just the latest — useful for resolving disputes about what was actually agreed at a given point in time.

When client and designer reach agreement, a **final summary** is drafted (referencing the last agreed checkpoint) and sent to the client for confirmation. Once confirmed, the project moves to the Design phase.

---

7. Design Generation

Once the final summary is confirmed, designs are produced against that final agreed summary (not per-checkpoint):

- **AI-generated designs**: The model produces 3-4 design options, each accompanied by structured specs (`specs_json`) — materials required, quantities, dimensions.
- **Designer-uploaded designs**: A designer can upload their own design; they manually fill in the same structured spec fields (dimensions, materials, quantities) via a form at upload time.

Both types are stored in a single **design gallery** for the client, each tagged by source (`ai_generated` / `designer_uploaded`), with a flag for which one was ultimately selected.

Regardless of source, every design converges on the same `specs_json` structure, so the quotation engine downstream doesn't need to care where the design came from.

---

8. Quotation Engine

The quotation is **not** an AI guess — it's a calculation based on company-supplied reference data.

Inputs maintained by admin/designer:

- **Materials/rates table** — item name, category (plywood, laminate, hardware, labor-per-sqft, etc.), unit cost, last updated date.
- **Margin configuration** — flat company-wide margin %, or category-wise margins (e.g., labor margin ≠ materials margin).

Calculation:

- Take the selected design's `specs_json` (materials, quantities, dimensions/labor estimate).
- Multiply against the materials/rates table.
- Apply the relevant margin(s).
- Produce a **structured breakdown** (not just a single number) — e.g., materials cost, labor cost, margin, total range (e.g., ₹1,00,000 – ₹1,20,000).

This breakdown is stored per quotation and can be edited/renegotiated without being regenerated from scratch.

---

9. Data Model (Draft)

Table	Key Fields
`users`	id, name, email, password_hash, role (`admin` / `designer`)
`clients`	id, name, contact_info, assigned_designer_id, current_phase, created_at

---

Table	Key Fields
`checkpoints`	id, client_id, summary_text, structured_diff, status (`draft` / `under_review` / `agreed`), source (`chat` / `manual`), created_at
`final summaries`	id, client_id, checkpoint_id_ref, confirmed_by (`client_reply` / `designer_override`), confirmed_at
`designs`	id, client_id, source (`ai_generated` / `designer_uploaded`), image_url, specs_json, is_selected
`materials_rates`	id, name, category, unit_cost, updated_at
`margin_config`	id, category, margin_percent
`quotations`	id, client_id, design_id, breakdown_json, total_min, total_max, status (`pending` / `approved`)
`audit_log`	id, user_id, action, entity_type, entity_id, timestamp

Notes:

- Admin sees all rows in `clients`. Designer sees `clients WHERE assigned_designer_id = self`.
- `audit_log` captures who created a checkpoint, who edited a quotation, who overrode a phase/confirmation — needed for accountability since multiple people touch the same client record.

---

10. System Architecture

- **Frontend:** Next.js — admin and designer dashboards, client list, client detail view with checkpoint timeline, design gallery, quotation view.
- **Backend:** FastAPI — single source of truth; exposes REST endpoints (`/clients`, `/checkpoints`, `/designs`, `/quotations`, `/materials-rates`, etc.).
- **Automation layer:** n8n — listens to conversations, generates summaries and diffs, and calls the FastAPI backend's endpoints to create/update checkpoints. n8n should **not** write directly to the database — everything flows through FastAPI to avoid two systems both claiming ownership of the data.

High-level flow:

Conversation (chat/call)
|
▼

n8n: fetch → summarize → diff

|

▼

FastAPI: POST /checkpoints

|

▼

Portal: checkpoint visible in timeline

|

▼

(repeat until agreement) → Final summary → Client confirms (reply or designer override)

|

▼

Phase: Discussion → Design

|

▼

Designs generated/uploaded (with specs_json) → one selected

|

▼

Phase: Design → Quotation/Approval

|

▼

Quotation calculated (specs_json × materials_rates × margin_config)

|

▼

Client approves → Phase: Quotation/Approval → Implementation

|

▼

Phase: Implementation → Completed/Payment

---

11. Open Questions / Decisions Still Needed

- Exact channel(s) for client conversation ingestion (WhatsApp, calls transcribed, email, etc.) — affects how n8n listens and parses.
- Whether "under_review" checkpoints need designer approval before becoming "agreed," or if that's automatic once client responds.
- How labor cost estimation works for designer-uploaded designs (manual entry vs. derived from dimensions).
- Whether margin config should be global or per-designer/per-category from day one, or added later.
- Authentication approach (JWT, session-based, etc.) and whether SSO is needed.

12. Suggested Build Order

1. **Database schema** — turn the draft above into real migrations (SQLModel/Alembic or similar).
2. **FastAPI backend** — core models + endpoints so n8n has something to call.
3. **n8n workflow** — trigger → summarize → diff → call backend.
4. **Next.js dashboard** — admin and designer views, client list, client detail with checkpoint timeline, design gallery, quotation view.
5. **Quotation engine logic** — calculation service consuming specs_json + materials_rates + margin_config.
6. **Auth & role-based access** — admin vs designer scoping.
